

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Final
Semester: Spring 2024

Duration: 1 Hour 40 Min
Full Marks: 35

CSE 470: Software Engineering

SET A

Name:

ID:

Section:

Q1. You've been tasked with designing a communication system for a multiplayer online game where players from around the world can interact in real-time. The system must support features such as player movement, chat messaging, and real-time updates of the state of the game across all connected clients. Each player's actions should be immediately reflected to others in the same game session.

- A. Considering the need for seamless, low-latency communication between the game clients and the game server, propose an architectural design that ensures efficient data transmission, synchronization, and responsiveness while accommodating a large number of concurrent players. **[Mark 3] [CO3]**
- B. Can the system face any disadvantages in using your chosen architecture? Explain probable disadvantages in detail. **[Mark 3] [CO3]**

Q2. Consider yourself a very Attentive student of BRAC University. You can study at university, and attend the classes at university. At this point, you want to take courses based on your choice during the Advising time. Sometimes you want to do advising with fewer credits considering the course load and sometimes you want to advise more courses to complete your graduation early. Having an open credit system for course advising allows you to choose courses and credit limits based on your interest. Now consider this advising process you and your other friends do the advising for the upcoming summer semester 2024. **(Methods can have only print statements as their body)**

- A. Considering the above scenario choose an Appropriate design pattern. Justify your answer. **[Mark 3] [CO3]**
- B. Write the pseudocode for the above scenario where you and your friend want to do the advising. **[Mark 3] [CO3]**

Q3. Consider the following code snippet below.

- A. Identify at least 3 code smells from the below code and name them **[Mark 2] [CO5]**
- B. Refactor the source code to remove 4 different Code smells. (write the code segment in your copy that has code smell and then show the correct way to refactor it). **[Mark 4] [CO5]**

<pre> import java.util.ArrayList; import java.util.List; public class Hospital { private List<Patient> patients; public Hospital() { this.patients = new ArrayList<>(); } public void registerPatient(Patient patient) { patients.add(patient); } public void updateHealthRecord(Patient patient, String condition) { patient.setHealthCondition(condition); } public void displayPatientDetails(String name,int age, String phoneNumber, String pId, boolean showBill) { for (Patient patient : patients) { if (patient.getName().equals(name)) { System.out.println("Patient Name: " + name); System.out.println("Health Condition: " + patient.getHealthCondition()); if (showBill) { System.out.println("Due Bill: " + patient.getDueBill()); } System.out.println("Contact No: " + phoneNumber); System.out.println("Patient Id: " + pId); System.out.println("-----"); return; } } System.out.println("Patient not found: " + name); } } public class Patient { private String name; public int age; public String phoneNumber; public String pId; private double dueBill; private String healthCondition; public Patient(String name, int age, String phoneNumber, String pId) { this.name = name; this.age = age; this.phoneNumber = phoneNumber; this.pId = pId; this.dueBill = 0; this.healthCondition = "Healthy"; } public void setHealthCondition(String healthCondition) { this.healthCondition = healthCondition; } } </pre>	<pre> public String getName() { return this.name; } public double getDueBill() { return this.dueBill; } public void checkValid(double amount) { //Validate the payment amount if (amount <= 0) { System.out.println("Error: Payment must be greater than zero."); return; } //Check if payment exceeds the due bill if (amount > dueBill) { System.out.println("Error: Payment amount exceeds the due bill."); return; } this.dueBill -= amount; //Generate a payment receipt System.out.println("Payment receipt for " + this.name); System.out.println("Amount Paid: \$" + amount); System.out.println("Remaining Due Bill: \$" + this.dueBill); } public String getHealthCondition() { return this.healthCondition; } public void addDueBill(double amount) { if (amount > 0) { this.dueBill += amount; } } } public class HealthcareSystem { public static void main(String[] args) { Hospital hospital = new Hospital(); Patient alice = new Patient("Alice", 25, "6663345", "234"); Patient bob = new Patient("Bob", 39, "3344567", "567"); hospital.registerPatient(alice); hospital.registerPatient(bob); hospital.updateHealthRecord(alice, "Flu"); alice.addDueBill(2000); hospital.updateHealthRecord(bob, "Diabetics"); bob.addDueBill(500); hospital.displayPatientDetails(alice.getName(), alice.age, alice.phoneNumber, alice.pId, true); hospital.displayPatientDetails(bob.getName(), bob.age, bob.phoneNumber, bob.pId, false); } } </pre>
---	--

Q4. Consider the following code snippet.

- Identify the nodes and draw the CFG of viewGamingPerformance() method. **[Mark 5] [CO4]**
- Identify all the path sets for the drawn CFG. **[Mark 4] [CO4]**
- Calculate the cyclomatic complexity of viewGamingPerformance() method. **[Mark 3] [CO4]**
- Write a test case for one path that has more than 4 nodes. **[Mark 2] [CO4]**
- Calculate SIX for Laptop class. **[Mark 3] [CO4]**

```
class PersonalComputer{
    public void startComputer(){
        System.out.println("Starting");
    }
    public void stopComputer(){
        System.out.println("Shutting down");
    }
    public void restartComputer(){
        System.out.println("Restarting");
    }
}
class GamingComputer extends PersonalComputer{
    public void playGames(){
        System.out.println("Playing");
    }
    public void boostPerformance(){
        System.out.println("Boosting");
    }
}
class Laptop extends PersonalComputer{
    public void stopComputer(){
        System.out.println("Laptop shutting down");
    }
    public void watchMovies(){
        System.out.println("Watching");
    }
}
class GamingLaptop extends GamingComputer{
    public void playGames(){
        System.out.println("Laptop Gaming");
    }
    public void savePower(){
        System.out.println("Saving power");
    }
}
```

```
public void boostPerformance(){
    System.out.println("PerformanceBoosted");
}
public void viewGamingPerformance(){
    int batteryLife = 100;
    boolean playingGames = true;
    boolean run = true;
    while(run){
        if(playingGames){
            int timePlayed = 0;
            while(timePlayed < batteryLife){
                timePlayed += 10;
                batteryLife -= 10;
                if(batteryLife > 50){
                    System.out.println("Save");
                }else{
                    System.out.println("Low!");
                    run = false;
                    playingGames = false;
                    System.out.println("Done");
                }
            }
        }else{
            System.out.println("No games");
        }
    }
    System.out.println("Thank You");
}
```